

```

<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>The Analytical Map – A Socratic Brainstorming Tool</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Fraunces:ital,opsz,wght@0,9
..144,300..900;1,9..144,300..900&family=Newsreader:ital,opsz,wght@0,6..72,
200..800;1,6..72,200..800&family=JetBrains+Mono:wght@400;500&display=swap"
rel="stylesheet">
<style>
  *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0;
}

:root {
  --bg: #ffffff;
  --bg-deep: #f5f5f5;
  --paper: #ffffff;
  --ink: #000000;
  --ink-soft: #2a2a2a;
  --ink-faded: #6a6a6a;
  --rule: #cccccc;
  --rule-soft: #e5e5e5;

  --c-issue: #7a1f1f;
  --c-time: #2d4a3e;
  --c-lang: #ad7a1a;
  --c-rhet: #2a3a6b;
  --c-author: #a8542f;
  --c-genre: #5d3a5f;

  --bg-tint-1: rgba(0, 0, 0, 0);
  --bg-tint-2: rgba(0, 0, 0, 0);

  --center-glow: #cccccc;
  --node-ring: #ffffff;

  --font-display: 'Fraunces', Georgia, 'Times New Roman', serif;
  --font-body: 'Newsreader', Georgia, 'Times New Roman', serif;
  --font-mono: 'JetBrains Mono', ui-monospace, 'SF Mono', Menlo,
monospace;
}

```

```

[data-theme="dark"] {
  --bg: #000000;
  --bg-deep: #0a0a0a;
  --paper: #000000;
  --ink: #ffffff;
  --ink-soft: #d5d5d5;
  --ink-faded: #959595;
  --rule: #333333;
  --rule-soft: #1a1a1a;

  --c-issue: #e85c5c;
  --c-time: #6db896;
  --c-lang: #e8a93a;
  --c-rhet: #7a9ce0;
  --c-author: #e8884a;
  --c-genre: #b08bb8;

  --bg-tint-1: rgba(0, 0, 0, 0);
  --bg-tint-2: rgba(0, 0, 0, 0);

  --center-glow: #333333;
  --node-ring: #000000;
}

html, body { min-height: 100vh; }
body {
  background: var(--bg);
  color: var(--ink);
  font-family: var(--font-body);
  font-size: 20px;
  line-height: 1.55;
  background-image:
    radial-gradient(ellipse at top left, var(--bg-tint-1), transparent
55%),
    radial-gradient(ellipse at bottom right, var(--bg-tint-2),
transparent 55%);
  background-attachment: fixed;
  overflow-x: hidden;
  transition: background 0.3s, color 0.3s;
}

.app { max-width: 1400px; margin: 0 auto; padding: 36px 40px 80px; }

.header { display: flex; justify-content: space-between; align-items:

```

```
flex-end; border-bottom: 1px solid var(--rule); padding-bottom: 22px;
margin-bottom: 32px; }
.h-left .title { font-family: var(--font-display);
font-variation-settings: 'opsz' 144, 'SOFT' 60; font-size: 52px;
font-weight: 380; letter-spacing: -0.025em; line-height: 0.95; font-style:
italic; color: var(--ink); }
.h-left .subtitle { font-family: var(--font-mono); font-size: 11px;
text-transform: uppercase; letter-spacing: 0.22em; color:
var(--ink-faded); margin-top: 10px; }
.h-right { display: flex; align-items: flex-end; gap: 18px; }
.h-right-text { font-family: var(--font-display); font-style: italic;
font-size: 14px; color: var(--ink-soft); text-align: right; line-height:
1.4; max-width: 240px; }
.theme-toggle {
background: transparent; border: 1px solid var(--rule); cursor:
pointer;
width: 38px; height: 38px; border-radius: 50%; color: var(--ink-soft);
font-size: 18px; line-height: 1; display: flex; align-items: center;
justify-content: center;
transition: all 0.2s; flex-shrink: 0;
font-family: var(--font-display);
}
.theme-toggle:hover { border-color: var(--ink); color: var(--ink);
transform: rotate(180deg); }

.prompt-section { background: var(--paper); border: 1px solid
var(--rule); padding: 24px 28px; margin-bottom: 32px; position: relative;
box-shadow: 4px 4px 0 var(--rule-soft); }
.prompt-label { font-family: var(--font-mono); font-size: 10px;
text-transform: uppercase; letter-spacing: 0.22em; color:
var(--ink-faded); margin-bottom: 12px; display: flex; align-items: center;
gap: 10px; }
.prompt-label::before { content: '$'; font-family: var(--font-display);
font-size: 14px; color: var(--c-issue); }
.prompt-textarea { width: 100%; background: transparent; border: none;
resize: vertical; font-family: var(--font-body); font-size: 19px;
line-height: 1.5; color: var(--ink); min-height: 50px; }
.prompt-textarea:focus { outline: none; }
.prompt-textarea::placeholder { color: var(--ink-faded); font-style:
italic; }
.prompt-actions { display: flex; justify-content: space-between;
align-items: center; margin-top: 16px; gap: 12px; flex-wrap: wrap; }
.prompt-meta { font-family: var(--font-mono); font-size: 10px;
text-transform: uppercase; letter-spacing: 0.18em; color:
var(--ink-faded); }
```

```

.btn-group { display: flex; gap: 10px; }

.btn-primary { background: var(--ink); color: var(--bg); border: none;
padding: 11px 26px; font-family: var(--font-mono); font-size: 11px;
text-transform: uppercase; letter-spacing: 0.22em; cursor: pointer;
transition: all 0.18s ease; }
  .btn-primary:hover:not(:disabled) { background: var(--c-issue);
transform: translate(-1px, -1px); box-shadow: 2px 2px 0 var(--ink); }
  .btn-primary:disabled { opacity: 0.35; cursor: not-allowed; }
  .btn-secondary { background: transparent; color: var(--ink-soft);
border: 1px solid var(--rule); padding: 10px 20px; font-family:
var(--font-mono); font-size: 11px; text-transform: uppercase;
letter-spacing: 0.22em; cursor: pointer; transition: all 0.15s; }
  .btn-secondary:hover { border-color: var(--ink); color: var(--ink); }

.main-grid { display: grid; grid-template-columns: minmax(0, 1.7fr)
minmax(0, 1fr); gap: 30px; align-items: start; }

.panel { background: var(--paper); border: 1px solid var(--rule);
padding: 26px 28px; box-shadow: 4px 4px 0 var(--rule-soft); position:
relative; }
  .panel-label { font-family: var(--font-mono); font-size: 10px;
text-transform: uppercase; letter-spacing: 0.22em; color:
var(--ink-faded); margin-bottom: 4px; display: flex; justify-content:
space-between; align-items: center; gap: 10px; }
  .panel-title { font-family: var(--font-display); font-style: italic;
font-size: 30px; font-weight: 700; line-height: 1; margin-bottom: 18px;
color: var(--ink); letter-spacing: -0.01em; }
  .panel-action { cursor: pointer; background: none; border: 1px solid
var(--rule); padding: 5px 11px; font-family: var(--font-mono); font-size:
9px; text-transform: uppercase; letter-spacing: 0.18em; color:
var(--ink-soft); transition: all 0.15s; white-space: nowrap; }
  .panel-action:hover:not(:disabled) { background: var(--ink); color:
var(--bg); border-color: var(--ink); }
  .panel-action:disabled { opacity: 0.4; cursor: not-allowed; }
  .panel-action.active { background: var(--c-rhet); color: var(--paper);
border-color: var(--c-rhet); }

.anchored-prompt { background: var(--bg); border-left: 3px solid
var(--c-issue); padding: 10px 14px; margin-bottom: 18px; font-family:
var(--font-display); font-style: italic; font-size: 14px; color:
var(--ink-soft); line-height: 1.5; }

.map-perspective { margin: 0 auto 12px; width: 100%; max-width: 580px;
overflow: visible; }

```

```

.map-wrap { position: relative; overflow: visible; }
.map-svg { width: 100%; aspect-ratio: 1; display: block; }

/* Flat node overlay – circles with labels inside */
.sphere-overlay { position: absolute; inset: 0; pointer-events: none; }
.sphere-group {
  position: absolute; width: 19%; aspect-ratio: 1;
  transform: translate(-50%, -50%);
  pointer-events: auto; cursor: pointer;
}
.sphere {
  position: relative;
  width: 100%; height: 100%; border-radius: 50%;
  background: var(--sphere-color);
  box-shadow: 0 2px 6px rgba(0,0,0,0.18);
  display: flex; align-items: center; justify-content: center;
  transition: transform 0.2s ease, box-shadow 0.2s ease;
  border: 2px solid rgba(255,255,255,0.15);
}
.sphere-label {
  color: #ffffff;
  font-family: var(--font-display);
  font-weight: 600;
  font-size: 13px;
  text-align: center;
  line-height: 1.15;
  padding: 0 6px;
  letter-spacing: -0.005em;
  user-select: none;
  pointer-events: none;
  text-shadow: 0 1px 2px rgba(0,0,0,0.3);
}
.sphere-group:hover .sphere { transform: scale(1.05); box-shadow: 0 4px
10px rgba(0,0,0,0.25); }
.sphere-group.is-selected .sphere {
  transform: scale(1.1);
  box-shadow: 0 0 0 3px var(--sphere-color), 0 6px 14px rgba(0,0,0,0.3);
}
.sphere-group.is-multi::after {
  content: ''; position: absolute; inset: -14%;
  border-radius: 50%; border: 2px dashed var(--sphere-color);
  pointer-events: none;
  animation: pulse-ring 1.6s infinite ease-in-out;
}
@keyframes pulse-ring {

```

```

    0%, 100% { opacity: 0.4; transform: scale(0.96); }
    50% { opacity: 0.95; transform: scale(1.10); }
}

/* Center analysis-concepts node – bigger, white, title inside */
.center-node {
    position: absolute;
    width: 27%; aspect-ratio: 1;
    left: 50%; top: 50%;
    transform: translate(-50%, -50%);
    border-radius: 50%;
    background: #ffffff;
    border: 2px solid #000000;
    display: flex; align-items: center; justify-content: center;
    flex-direction: column;
    pointer-events: none;
    box-shadow: 0 4px 14px rgba(0,0,0,0.12);
    z-index: 2;
}

.center-node-text {
    font-family: var(--font-display);
    font-style: italic;
    font-weight: 700;
    font-size: 17px;
    color: #000000;
    text-align: center;
    line-height: 1.1;
    letter-spacing: -0.015em;
}

/* Question bubbles – branch off the selected concept */
.question-bubble {
    position: absolute;
    width: 11%; aspect-ratio: 1;
    transform: translate(-50%, -50%);
    border-radius: 50%;
    background: var(--paper);
    border: 2px solid var(--bubble-color);
    cursor: pointer;
    pointer-events: auto;
    display: flex; align-items: center; justify-content: center;
    transition: transform 0.2s ease, background 0.2s ease;
    z-index: 5;
    box-shadow: 0 2px 6px rgba(0,0,0,0.12);
}

```

```

    .question-bubble:hover { transform: translate(-50%, -50%) scale(1.1); }
    .question-bubble.is-active { background: var(--bubble-color); transform:
translate(-50%, -50%) scale(1.15); z-index: 7; }
    .question-bubble.is-added { background: var(--c-time); border-color:
var(--c-time); }
    .question-bubble-inner {
        font-family: var(--font-display);
        font-style: italic;
        font-weight: 700;
        font-size: 15px;
        color: var(--bubble-color);
        pointer-events: none;
    }
    .question-bubble.is-active .question-bubble-inner,
    .question-bubble.is-added .question-bubble-inner { color: #ffffff; }

/* Deeper question bubbles – branch off a question bubble */
    .deeper-bubble {
        position: absolute;
        width: 8%; aspect-ratio: 1;
        transform: translate(-50%, -50%);
        border-radius: 50%;
        background: var(--paper);
        border: 1.5px dashed var(--bubble-color);
        cursor: pointer;
        pointer-events: auto;
        display: flex; align-items: center; justify-content: center;
        transition: transform 0.2s ease, background 0.2s ease;
        z-index: 4;
        box-shadow: 0 2px 5px rgba(0,0,0,0.1);
    }
    .deeper-bubble:hover { transform: translate(-50%, -50%) scale(1.12);
border-style: solid; }
    .deeper-bubble.is-active { background: var(--bubble-color);
border-style: solid; transform: translate(-50%, -50%) scale(1.15);
z-index: 7; }
    .deeper-bubble.is-added { background: var(--c-time); border-color:
var(--c-time); border-style: solid; }
    .deeper-bubble-inner {
        font-family: var(--font-mono);
        font-size: 10px;
        font-weight: 500;
        color: var(--bubble-color);
        pointer-events: none;
        letter-spacing: 0;
    }

```

```

}
.deeper-bubble.is-active .deeper-bubble-inner,
.deeper-bubble.is-added .deeper-bubble-inner { color: #ffffff; }

/* Floating panel showing question text + actions */
.qpanel {
  position: absolute;
  width: 280px;
  background: var(--paper);
  border: 1px solid var(--rule);
  border-radius: 8px;
  padding: 14px 16px;
  box-shadow: 0 10px 26px rgba(0,0,0,0.16), 0 3px 8px rgba(0,0,0,0.08);
  z-index: 20;
  transform: translate(-50%, -50%);
  pointer-events: auto;
  animation: callout-pop 0.2s ease-out;
}
@keyframes callout-pop {
  from { opacity: 0; transform: translate(-50%, -50%) scale(0.85); }
  to { opacity: 1; transform: translate(-50%, -50%) scale(1); }
}
.qpanel-close {
  position: absolute; top: 6px; right: 8px;
  background: none; border: none; cursor: pointer;
  font-size: 18px; color: var(--ink-faded); padding: 0;
  line-height: 1; width: 20px; height: 20px;
  display: flex; align-items: center; justify-content: center;
}
.qpanel-close:hover { color: var(--ink); }
.qpanel-meta {
  font-family: var(--font-mono);
  font-size: 9px;
  text-transform: uppercase;
  letter-spacing: 0.18em;
  color: var(--ink-faded);
  margin-bottom: 6px;
  padding-right: 24px;
}
.qpanel-text {
  font-family: var(--font-body);
  font-size: 15px;
  line-height: 1.45;
  color: var(--ink);
  margin-bottom: 14px;
}

```

```

}
.qpanel-actions { display: flex; gap: 8px; flex-wrap: wrap; }
.qpanel-loading { font-family: var(--font-display); font-style: italic;
color: var(--ink-faded); padding: 10px; text-align: center; font-size:
13px; }
.qpanel-error { font-family: var(--font-mono); font-size: 10px; color:
var(--c-issue); text-align: center; padding: 8px; text-transform:
uppercase; letter-spacing: 0.1em; }
.map-node-circle { transition: all 0.28s cubic-bezier(0.4, 0, 0.2, 1); }
.map-node-group { cursor: pointer; }
.map-node-label { font-family: var(--font-display); font-size: 13px;
font-weight: 500; fill: var(--ink); pointer-events: none; user-select:
none; letter-spacing: -0.005em; }
.map-link-hit { cursor: pointer; }
.map-center-text { font-family: var(--font-display); font-style: italic;
font-size: 11px; fill: var(--ink-faded); pointer-events: none;
user-select: none; }

.link-builder { background: var(--bg); border: 1px dashed var(--c-rhet);
padding: 10px 14px; margin-bottom: 12px; display: flex; align-items:
center; gap: 10px; flex-wrap: wrap; }
.link-builder-label { font-family: var(--font-mono); font-size: 10px;
text-transform: uppercase; letter-spacing: 0.18em; color: var(--c-rhet); }
.link-builder-tag { display: inline-block; font-family:
var(--font-mono); font-size: 10px; padding: 3px 8px; color: var(--paper);
}
.link-builder-buttons { margin-left: auto; display: flex; gap: 8px; }

.connection-hint { font-family: var(--font-mono); font-size: 10px;
text-transform: uppercase; letter-spacing: 0.2em; text-align: center;
margin-top: 4px; min-height: 16px; color: var(--c-rhet); }
.connection-hint.idle { color: var(--ink-faded); }

.questions-section { margin-top: 24px; padding-top: 24px; border-top:
1px solid var(--rule-soft); }
.selected-tags { display: flex; align-items: center; flex-wrap: wrap;
gap: 8px; margin-bottom: 4px; }
.selected-tag { display: inline-block; font-family: var(--font-mono);
font-size: 9px; text-transform: uppercase; letter-spacing: 0.22em; color:
var(--paper); padding: 4px 10px; }
.tag-x { font-family: var(--font-display); font-style: italic; color:
var(--ink-faded); }
.selected-description { font-family: var(--font-display); font-style:
italic; font-size: 14px; color: var(--ink-soft); line-height: 1.5;
margin-bottom: 22px; }

```

```

    .questions-list { display: flex; flex-direction: column; gap: 14px; }
    .question-item { padding: 16px 18px; background: var(--bg); border-left:
3px solid var(--ink-soft); font-family: var(--font-body); font-size: 16px;
line-height: 1.45; position: relative; transition: all 0.15s; color:
var(--ink); }
    .question-item:hover { background: var(--bg-deep); }
    .question-number { font-family: var(--font-display); font-style: italic;
font-size: 13px; color: var(--ink-faded); margin-right: 8px; font-weight:
500; }
    .question-add { margin-top: 10px; background: none; border: none;
font-family: var(--font-mono); font-size: 10px; text-transform: uppercase;
letter-spacing: 0.18em; color: var(--ink-faded); cursor: pointer; padding:
0; transition: color 0.15s; }
    .question-add:hover:not(:disabled) { color: var(--ink); }
    .question-add:disabled { cursor: default; }
    .question-add.added { color: var(--c-time); font-style: italic; }

    .loading-msg { font-family: var(--font-display); font-style: italic;
color: var(--ink-faded); padding: 28px 14px; text-align: center;
font-size: 15px; }
    .loading-msg::after { content: '...'; display: inline-block; animation:
dots 1.4s infinite steps(4); width: 0; overflow: hidden; vertical-align:
bottom; }
    @keyframes dots { 0%, 25% { width: 0; } 50% { width: 0.4em; } 75% {
width: 0.8em; } 100% { width: 1.2em; } }
    .error-msg { font-family: var(--font-mono); font-size: 11px; color:
var(--c-issue); text-align: center; padding: 16px; text-transform:
uppercase; letter-spacing: 0.12em; }

    .outline-empty { font-family: var(--font-display); font-style: italic;
color: var(--ink-faded); font-size: 16px; text-align: center; padding:
60px 20px; line-height: 1.6; }
    .outline-empty::before, .outline-empty::after { content: '—'; color:
var(--rule); font-style: normal; }
    .outline-empty::after { content: ' —'; }
    .outline-item { background: var(--bg); border-left: 3px solid
var(--ink-soft); padding: 16px 18px 14px; margin-bottom: 14px; position:
relative; }
    .outline-source { font-family: var(--font-mono); font-size: 9px;
text-transform: uppercase; letter-spacing: 0.22em; margin-bottom: 8px;
display: flex; align-items: center; gap: 8px; }
    .outline-source .roman { font-family: var(--font-display); font-style:
italic; color: var(--ink-faded); text-transform: none; letter-spacing: 0;
font-size: 13px; }

```

```

.outline-question-text { font-size: 15.5px; line-height: 1.4;
margin-bottom: 12px; color: var(--ink); }
.outline-notes { width: 100%; background: var(--paper); border: 1px
solid var(--rule-soft); padding: 10px 12px; font-family: var(--font-body);
font-size: 14.5px; line-height: 1.5; resize: vertical; min-height: 60px;
color: var(--ink); transition: border-color 0.15s; }
.outline-notes:focus { outline: none; border-color: var(--ink-soft); }
.outline-notes::placeholder { color: var(--ink-faded); font-style:
italic; }
.outline-controls { display: flex; gap: 14px; margin-top: 8px;
align-items: center; }
.outline-mini-btn { background: none; border: none; font-family:
var(--font-mono); font-size: 9px; text-transform: uppercase;
letter-spacing: 0.18em; color: var(--ink-faded); cursor: pointer; padding:
0; transition: color 0.15s; }
.outline-mini-btn:hover:not(:disabled) { color: var(--ink); }
.outline-mini-btn:disabled { opacity: 0.3; cursor: not-allowed; }
.outline-mini-btn.remove:hover { color: var(--c-issue); }

.colophon { text-align: center; margin-top: 60px; font-family:
var(--font-display); font-style: italic; font-size: 13px; color:
var(--ink-faded); }
.colophon::before { content: '◆'; display: block; color: var(--rule);
margin-bottom: 10px; font-size: 12px; }

@media (max-width: 1080px) { .main-grid { grid-template-columns: 1fr; }
.map-wrap { max-width: 460px; } }
@media (max-width: 640px) {
  .app { padding: 22px 18px; }
  .h-left .title { font-size: 36px; }
  .h-right-text { display: none; }
  .header { flex-direction: row; align-items: center; }
  .panel { padding: 20px; }
  .prompt-actions { flex-direction: column; align-items: stretch; }
}
</style>
</head>
<body>
<div id="app"></div>

<script>
// =====
// THE ANALYTICAL MAP – vanilla JS
// =====

```

```

const NODES_BASE = [
  { id: 'issue', label: 'Global Issue', angle: -90, description:
    'the broader real-world concerns the text engages with – power, identity,
    belonging, justice, knowledge, the human condition.' },
  { id: 'time', label: 'Time & Space', angle: -30, description:
    'historical context, setting, geography, chronology – when and where the
    text exists, and the worlds it constructs.' },
  { id: 'lang', label: 'Language', angle: 30, description:
    'diction, syntax, imagery, figurative language, sound – how words behave
    at the level of the sentence.' },
  { id: 'rhet', label: 'Rhetoric', angle: 90, description:
    'persuasion, argument, audience, appeals – how the text positions and
    moves its reader.' },
  { id: 'author', label: 'Authorial Choices', angle: 150, description:
    'deliberate craft decisions – structure, point of view, what is included
    and omitted, narrative voice, form.' },
  { id: 'genre', label: 'Genre', angle: 210, description:
    'conventions, expectations, traditions – and how the text follows, bends,
    hybridizes, or breaks them.' }
];

function nodeColor(id) {
  return
  getComputedStyle(document.documentElement).getPropertyValue('--c-' +
  id).trim() || '#000';
}

const CENTER = 220, RADIUS = 138, VIEWBOX = 440, LABEL_OFFSET = 64;
const NODE_BY_ID = {};
NODES_BASE.forEach(function(n) { NODE_BY_ID[n.id] = n; });
const ROMAN =
['i','ii','iii','iv','v','vi','vii','viii','ix','x','xi','xii','xiii','xiv',
',','xv','xvi','xvii','xviii','xix','xx','xxi','xxii','xxiii','xxiv','xxv'];

function defaultNodePositions() {
  const pos = {};
  NODES_BASE.forEach(function(n) {
    const r = (n.angle * Math.PI) / 180;
    pos[n.id] = {
      x: CENTER + RADIUS * Math.cos(r),
      y: CENTER + RADIUS * Math.sin(r)
    };
  });
  return pos;
}

```

```
// labelOffsetFor: given a node's current position, compute label position
offset outward from center
function labelPosFor(p) {
  const dx = p.x - CENTER;
  const dy = p.y - CENTER;
  const dist = Math.sqrt(dx*dx + dy*dy) || 1;
  const out = dist + LABEL_OFFSET;
  return { lx: CENTER + (dx/dist) * out, ly: CENTER + (dy/dist) * out };
}
```

```
// Static fallback questions (used when no prompt anchored, or when API
fails)
```

```
const STATIC_QUESTIONS = {
  'issue': [
    "What real-world concern does this text seem most invested in, even
when it isn't saying so directly?",
    "Whose suffering or pleasure does the text take seriously, and whose
does it leave on the margins?",
    "If you had to name the central tension or anxiety driving the text,
how would you describe it?",
    "How would your reading change if you assumed the issue at the heart
of the text were the opposite of what you first thought?",
    "Where do the text's concerns push against the values of the world it
was made in?"
  ],
  'time': [
    "What does the text choose to dwell on for a long time, and what does
it skim past – and what does that pacing reveal?",
    "What relationship does the text's setting have with the era it was
written in: faithful, distorted, dreamlike, accusatory?",
    "Where in the text do you feel time slow down, speed up, or fold back
– and what work is that doing?",
    "What kinds of spaces are off-limits in this text, and what kinds are
dwelt in?",
    "If you transplanted this text to another time or place, what
specifically would break?"
  ],
  'lang': [
    "What word or image does the text return to – and what changes each
time it returns?",
    "Which sentences sound nothing like the ones around them, and what
makes them stand apart?",
    "Where does the language conceal as much as it reveals?",
    "What happens at the level of sound – rhythm, repetition, silence –
```

```

that isn't happening at the level of meaning?",
    "If you stripped away one figurative device the text leans on, what
    would collapse?"
],
    'rhet': [
        "Who does the text imagine its reader to be, and how can you tell?",
        "What does the text want from you – belief, doubt, complicity,
        discomfort?",
        "Where does the text try hardest to persuade, and what does that
        effort reveal about what is at stake?",
        "What does the text refuse to argue for explicitly, and how does it
        carry that argument anyway?",
        "Where are you being moved before you have time to decide whether to
        be moved?"
    ],
    'author': [
        "What does the author leave out that another writer might have
        included – and what does that omission accomplish?",
        "Whose voice is missing from the text, and is that absence accidental
        or designed?",
        "Where does the author make a choice that feels difficult or risky,
        and what is gained by taking that risk?",
        "If you imagine the text written from a different point of view, what
        would you suddenly know that you do not know now?",
        "What does the structure of the text – its order, its proportions, its
        endings – tell you about what the author wants you to feel last?"
    ],
    'genre': [
        "What promises does this genre make to its reader, and which ones does
        this text keep, break, or twist?",
        "What does this text borrow from outside its own genre – and what is
        being smuggled in by that borrowing?",
        "Where does the text obey genre convention so closely that the
        obedience itself becomes the point?",
        "What would be impossible to say in this text if the genre were
        different?",
        "What does the text reveal about its genre that you couldn't know
        without reading it?"
    ],
    // pairs
    'issue::time': ["How does the historical moment the text exists in shape
    what even counts as the issue?", "Where in the text does the setting carry
    the weight of the global concern – as if the place itself were arguing?",
    "If you shifted the issue to a different era, what would the text have to
    abandon to still make sense?"]],

```

'issue::lang': ["What word does the text most depend on to make its concern feel urgent – and what happens if you remove that word?", "Where does the language of the text contradict its stated concern, and what does that gap reveal?", "How does the text's vocabulary betray which side of the global issue it has already chosen?"],

'issue::rhet': ["What is the text trying to convince you the issue is, before you have time to ask?", "Whose argument about this issue does the text refuse to even acknowledge, and what does that silence accomplish?", "Where does the text use feeling to do work that an argument could not?"],

'author::issue': ["What does the author's choice of perspective force you to care about – and whose stake in the issue does it obscure?", "Where does an authorial decision about structure or omission make the issue more uncomfortable than the content alone would?", "If the author had taken the safer choice at any of these moments, what would the issue lose?"],

'genre::issue': ["What does this genre traditionally allow itself to say about issues like this – and what does it tend to avoid?", "How is the text using the form's conventions to slip past your defenses about the issue?", "Where does the genre seem inadequate to the weight of the issue – and is the text aware of that inadequacy?"],

'lang::time': ["Which words feel out of place in the time the text inhabits, and what is the text doing by misplacing them?", "How does the rhythm of the language match – or refuse to match – the pacing of the world it builds?", "What does the language preserve from a particular era that the rest of the text seems to want to escape?"],

'rhet::time': ["Who is the text trying to reach across the distance of its own setting?", "How does the text use historical or geographic distance as part of its persuasion – making the strange feel obvious, or the familiar feel strange?", "What argument can only be made in this specific time and place?"],

'author::time': ["What does the author know about this time or place that the characters cannot, and how does that asymmetry shape your reading?", "Where does the author's choice of when to begin and when to end the story do the most work?", "If the author moved one scene to a different location, what would shift in what the text is doing?"],

'genre::time': ["How does this genre typically handle the era or setting the text occupies – and where does this text break with that?", "What is the text smuggling across time by hiding it inside a familiar form?", "Where does the genre's idea of 'where a story happens' feel invented for, or against, this setting?"],

'lang::rhet': ["Which single sentence is doing the most persuasive work – and what about its construction makes that possible?", "Where does the language stop trying to convince and start trying to enchant?", "How does the smallest unit – a word, a piece of punctuation – carry an argument larger than itself?"],

'author::lang': ["What is one decision at the sentence level that, if

reversed, would change your sense of who is speaking?", "Where does the author resist a fluent or natural phrasing – and what is gained in the resistance?", "What does the author's diction reveal about whose ear they are writing for?"]],

'genre::lang': ["What kinds of sentences does this genre teach you to expect – and where does the text break that expectation?", "Where does the language sound like another genre intruding, and what does that intrusion do?", "What can this genre's language not yet say, and where does the text strain at that limit?"]],

'author::rhet': ["What persuasive effect is the author chasing, and what choices have they made to achieve it without saying so?", "Where does the author's structural decision – order, withholding, repetition – do the work of an argument?", "What is the author trying to keep you from concluding too quickly?"]],

'genre::rhet': ["What does this genre let an author argue for that another form could not?", "Where does the text rely on genre convention to do persuasive work for it – and where does it have to fight the convention?", "What kind of reader does this genre presume, and how does the text confirm or unsettle that presumption?"]],

'author::genre': ["Where is the author working with the genre, and where are they working against it?", "What does the author preserve from the genre's tradition, and what does that preservation tell you about their stance?", "If you removed the genre's expectations from your reading entirely, which authorial choices would suddenly become strange?"]
};

// State

```
const state = {
  theme: 'light',
  promptDraft: '',
  prompt: '',
  selected: null,      // { type: 'node'|'link', id }
  links: [],           // [{ key, nodes: [ids] }]
  linkMode: false,
  linkBuilding: [],    // node IDs currently being selected for new
link
  multiSelection: [],  // node IDs selected via shift+click for ad-hoc
hybrid
  nodePositions: defaultNodePositions(), // { id: {x, y} } – draggable
positions
  outline: [],          // [{ id, question, sourceLabel, color, notes,
qKey }]
  addedKeys: {},
  questionCache: {},    // { key: [questions] }
  questionLoading: {},  // { key: bool }
```

```

    questionError: {},          // { key: string }
    activeQuestionKey: null,    // 'conceptId::idx' - currently opened question
bubble
    deeperQuestions: {},        // { qKey: [generated deeper questions] }
    deeperLoading: {},          // { qKey: bool }
    deeperError: {},            // { qKey: string }
    activeDeeperKey: null       // 'qKey::idx' - currently opened deeper bubble
};

function esc(s) {
    return String(s == null ? '' : s).replace(/[\<>'"]/g, function(c) {
        return
    ({'&': '&', '<': '<', '>': '>', '"': '"', "'": '''})[c];
    });
}

function $(id) { return document.getElementById(id); }

// =====
// QUESTION GENERATION - API + static fallback
// =====

function extractQuestions(text) {
    const cleaned = (text || '').replace(/```json|```/g, '').trim();
    try {
        const j = JSON.parse(cleaned);
        if (Array.isArray(j)) return j.filter(function(s) { return typeof s
=== 'string' && s.length > 5; });
        if (j && Array.isArray(j.questions)) return j.questions;
    } catch (e) {}
    const m = cleaned.match(/\[[\s\S]*\]/);
    if (m) {
        try {
            const j = JSON.parse(m[0]);
            if (Array.isArray(j)) return j.filter(function(s) { return typeof s
=== 'string' && s.length > 5; });
        } catch (e) {}
    }
    const lines = cleaned.split('\n').map(function(l) { return l.trim(); })
        .filter(function(l) { return l.length > 10; })
        .map(function(l) { return l.replace(/^[\d]+[\.\.]\s*/,
''.replace(/^[^-\.*]\s*/, '').replace(/^[["]|["']$/g, '').trim(); })
        .filter(function(l) { return l.endsWith('?'); });
    if (lines.length >= 2) return lines;
    return null;
}

```

```

async function callAPI(apiPrompt) {
  const res = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "claude-sonnet-4-5",
      max_tokens: 1024,
      messages: [{ role: "user", content: apiPrompt }]
    })
  });
  if (!res.ok) throw new Error('API ' + res.status);
  const data = await res.json();
  const text = (data.content || []).map(function(c) { return c.text || '';
}).join('');
  const qs = extractQuestions(text);
  if (!qs || qs.length === 0) throw new Error('No questions extracted');
  return qs;
}

function buildAPIPrompt(nodeIds) {
  const nodes = nodeIds.map(function(id) { return NODE_BY_ID[id]; });
  const labels = nodes.map(function(n) { return n.label; });
  const descList = nodes.map(function(n) { return '- **' + n.label + '** -
' + n.description; }).join('\n');
  const count = nodeIds.length === 1 ? 5 : (nodeIds.length === 2 ? 4 : 3);
  const intersectionPhrase = nodeIds.length === 1
    ? 'the analytical concept of **' + labels[0] + '**'
    : 'the intersection of these ' + nodeIds.length + ' analytical
concepts (every question must meaningfully involve ALL of them, not just
one)';
  const promptCtx = state.prompt
    ? 'The student is working with this specific prompt or text
focus:\n\n"' + state.prompt + '"\n\nThe questions you generate must be
DIRECTLY tailored to this prompt – reference the text, author, themes, or
topic the student named where appropriate.'
    : 'The student has not yet anchored a specific text. Generate
questions that work for any literary text, framed around "the text".';

  return 'You are a Socratic tutor helping a student brainstorm a literary
analysis essay.\n\n' +
    promptCtx + '\n\n' +
    'Generate ' + count + ' deep, open-ended Socratic questions that probe
' + intersectionPhrase + ':\n' +
    descList + '\n\n' +

```

```

    'Strict rules:\n' +
    '- DO NOT provide content, answers, interpretations, or hints toward
an interpretation.\n' +
    '- DO NOT name specific texts, authors, or themes unless the student
already mentioned them in the prompt above.\n' +
    '- Each question must provoke deep analytical thinking – it should NOT
be answerable with quick recall.\n' +
    '- Help the student notice what they might overlook.\n' +
    '- Address the student in the second person ("you").\n' +
    '- Keep each question under 35 words.\n' +
    '- Vary the question type: descriptive, evaluative, comparative,
counterfactual, structural.\n' +
    (nodeIds.length > 1 ? '- Each question MUST meaningfully involve ALL '
+ nodeIds.length + ' concepts together.\n' : '') +
    '\nReturn ONLY a valid JSON array of exactly ' + count + ' question
strings. No preamble, no markdown, no code fences.\nExample format:
["Question one?", "Question two?", ...]';
}

```

```

function getNodeIdsForKey(key) {
  // key is either a single node id or sorted ids joined by '::'
  return key.indexOf('::') >= 0 ? key.split('::') : [key];
}

```

```

async function fetchQuestionsFor(key) {
  if (state.questionCache[key]) return;
  if (state.questionLoading[key]) return;

```

```

  const nodeIds = getNodeIdsForKey(key);

```

```

  // If no prompt is anchored, use static questions immediately
  if (!state.prompt && STATIC_QUESTIONS[key]) {
    state.questionCache[key] = STATIC_QUESTIONS[key];
    render();
    return;
  }

```

```

  // Otherwise try API
  state.questionLoading[key] = true;
  state.questionError[key] = null;
  render();

```

```

  try {
    const apiPrompt = buildAPIPrompt(nodeIds);
    const qs = await callAPI(apiPrompt);

```

```

        state.questionCache[key] = qs;
    } catch (err) {
        console.error('Question generation failed:', err);
        // Fall back to static if available
        if (STATIC_QUESTIONS[key]) {
            state.questionCache[key] = STATIC_QUESTIONS[key];
            state.questionError[key] = 'Live generation unavailable - showing
universal questions.';
        } else {
            state.questionError[key] = 'Could not generate questions for this
combination. (Live generation requires the artifact preview
environment.)';
        }
    } finally {
        state.questionLoading[key] = false;
        render();
    }
}

// =====
// BUILDERS
// =====

function buildHeader() {
    const themeIcon = state.theme === 'light' ? '☐' : '◼';
    const themeLabel = state.theme === 'light' ? 'Switch to dark mode' :
'Switch to light mode';
    return ''
        + '<header class="header">'
        + '  <div class="h-left">'
        +    '<div class="title">The Analytical Map</div>'
        +    '<div class="subtitle">A Socratic brainstorming
instrument</div>'
        +  '</div>'
        +  '<div class="h-right">'
        +    '<div class="h-right-text">Six concepts. <br>Their
intersections. <br><em style="color: var(--c-issue);">Your</em>
thinking.</div>'
        +    '<button class="theme-toggle" data-action="toggle-theme"
title="' + themeLabel + '" aria-label="' + themeLabel + '">' + themeIcon +
'</button>'
        +  '</div>'
        + '</header>';
}

```

```

function buildPromptSection() {
  const hasPrompt = !!state.prompt;
  const meta = hasPrompt
    ? ('Anchored · ' + state.prompt.length + ' characters · questions are
now tailored to your prompt')
    : 'Optional – anchors the questions to your specific essay. Or just
start clicking nodes below for universal questions.';
  return ''
    + '<section class="prompt-section">'
    + '  <div class="prompt-label">Your essay prompt or text focus</div>'
    + '  <textarea id="prompt-textarea" class="prompt-textarea"
placeholder="Paste your essay prompt, name the text and author you\'re
analyzing, or describe what you\'re writing about..." rows="2"></textarea>'
    + '  <div class="prompt-actions">'
    + '    <div class="prompt-meta" id="prompt-meta">' + meta + '</div>'
    + '    <div class="btn-group">'
    + '      (hasPrompt ? <button class="btn-secondary"
data-action="reset">Clear all</button>' : '')
    + '      <button class="btn-primary" data-action="begin">' +
(hasPrompt ? 'Re-anchor' : 'Anchor prompt') + '</button>'
    + '    </div>'
    + '  </div>'
    + '</section>';
}

```

```

function buildNodeLabel(node, isSel) {
  const color = nodeColor(node.id);
  const style = isSel
    ? 'font-style: italic; font-weight: 600; fill: ' + color + ';'
    : 'font-style: normal; font-weight: 500; fill: var(--ink);';
  const label = node.label;
  const pos = state.nodePositions[node.id];
  const lp = labelPosFor(pos);
  let inner;
  if (label.indexOf(' & ') !== -1) {
    const parts = label.split(' & ');
    inner = parts.map(function(p, i) {
      return '<tspan x="' + lp.lx + '" dy="' + (i === 0 ? '-0.5em' :
'1.1em') + '">' + esc(p) + (i === 0 ? ' & ' : '') + '</tspan>';
    }).join('');
  } else if (label.indexOf(' ') !== -1 && label !== 'Global Issue') {
    const parts = label.split(' ');
    inner = parts.map(function(p, i) {
      const dy = i === 0 ? ('-' + ((parts.length - 1) * 0.55) + 'em') :
'1.1em';

```

```

        return '<tspan x="' + lp.lx + '" dy="' + dy + '">' + esc(p) +
'</tspan>';
    }).join('');
} else {
    inner = esc(label);
}
return '<text x="' + lp.lx + '" y="' + lp.ly + '" text-anchor="middle"
dominant-baseline="middle" class="map-node-label" style="' + style + '">'
+ inner + '</text>';
}

function buildSVG() {
    const pos = state.nodePositions;
    let svg = '<svg viewBox="0 0 ' + VIEWBOX + ' ' + VIEWBOX + '"
class="map-svg">';

    // ---- Gradient defs for 3D sphere effect ----
    svg += '<defs>';
    svg += '<radialGradient id="centerGlow" cx="50%" cy="50%" r="50%"><stop
offset="0%" stop-color="' +
getComputedStyle(document.documentElement).getPropertyValue('--center-glow
').trim() + '" stop-opacity="0.3"/><stop offset="100%" stop-color="' +
getComputedStyle(document.documentElement).getPropertyValue('--center-glow
').trim() + '" stop-opacity="0"/></radialGradient>';
    // Highlight (top-left bright spot)
    svg += '<radialGradient id="sphereHighlight" cx="32%" cy="30%"
r="55%">';
    svg += '<stop offset="0%" stop-color="#ffffff" stop-opacity="0.78"/>';
    svg += '<stop offset="45%" stop-color="#ffffff" stop-opacity="0.18"/>';
    svg += '<stop offset="100%" stop-color="#ffffff" stop-opacity="0"/>';
    svg += '</radialGradient>';
    // Edge shading (darker rim)
    svg += '<radialGradient id="sphereShade" cx="50%" cy="50%" r="55%">';
    svg += '<stop offset="55%" stop-color="#000000" stop-opacity="0"/>';
    svg += '<stop offset="100%" stop-color="#000000" stop-opacity="0.4"/>';
    svg += '</radialGradient>';
    // Drop shadow (under sphere)
    svg += '<radialGradient id="sphereDrop" cx="50%" cy="50%" r="50%">';
    svg += '<stop offset="0%" stop-color="#000000" stop-opacity="0.35"/>';
    svg += '<stop offset="100%" stop-color="#000000" stop-opacity="0"/>';
    svg += '</radialGradient>';
    svg += '</defs>';

    svg += '<circle cx="' + CENTER + '" cy="' + CENTER + '" r="' + (RADIUS -
50) + '" fill="url(#centerGlow)"/>';

```

```

// ---- Connecting lines between every pair of nodes (the web) ----
for (let i = 0; i < NODES_BASE.length; i++) {
  for (let j = i + 1; j < NODES_BASE.length; j++) {
    const a = pos[NODES_BASE[i].id], b = pos[NODES_BASE[j].id];
    svg += '<line x1="' + a.x + '" y1="' + a.y + '" x2="' + b.x + '"
y2="' + b.y + '" stroke="var(--ink-faded)" stroke-width="0.5"
opacity="0.25" pointer-events="none"/>';
  }
}

// ---- Perimeter hex outline (nodes in angular order) ----
const ringOrder = NODES_BASE.slice().sort(function(a, b) { return
a.angle - b.angle; });
const ringPts = ringOrder.map(function(n) { const p = pos[n.id]; return
p.x + ',' + p.y; }).join(' ');
svg += '<polygon points="' + ringPts + '" fill="none"
stroke="var(--ink-soft)" stroke-width="0.8" stroke-dasharray="3 4"
opacity="0.5" pointer-events="none"/>';

// ---- Spokes from center to each node ----
NODES_BASE.forEach(function(n) {
  const p = pos[n.id];
  svg += '<line x1="' + CENTER + '" y1="' + CENTER + '" x2="' + p.x + '"
y2="' + p.y + '" stroke="var(--ink-faded)" stroke-width="0.5"
stroke-dasharray="1 3" opacity="0.35" pointer-events="none"/>';
});

svg += '<circle cx="' + CENTER + '" cy="' + CENTER + '" r="4"
fill="var(--ink-faded)"/>';

// ---- Saved links ----
state.links.forEach(function(link) {
  const isSel = state.selected && state.selected.type === 'link' &&
state.selected.id === link.key;
  const sortedNodes = link.nodes.slice().sort(function(a, b) { return
NODE_BY_ID[a].angle - NODE_BY_ID[b].angle; });
  const stroke = isSel ? 'var(--ink)' : 'var(--ink)';
  const sw = isSel ? 3 : 2;
  const dash = isSel ? '0' : '0';

  if (sortedNodes.length === 2) {
    const a = pos[sortedNodes[0]], b = pos[sortedNodes[1]];
    svg += '<g>';
    svg += '<line class="map-link-hit" data-action="link" data-id="' +

```

```

esc(link.key) + '" x1="' + a.x + '" y1="' + a.y + '" x2="' + b.x + '"
y2="' + b.y + '" stroke="transparent" stroke-width="14"/>';
    svg += '<line x1="' + a.x + '" y1="' + a.y + '" x2="' + b.x + '"
y2="' + b.y + '" stroke="' + stroke + '" stroke-width="' + sw + '"
opacity="' + (isSel ? 1 : 0.75) + '" pointer-events="none"/>';
    svg += '</g>';
    } else {
        const pts = sortedNodes.map(function(id) { const p2 = pos[id];
return p2.x + ',' + p2.y; }).join(' ');
        svg += '<g>';
        svg += '<polygon class="map-link-hit" data-action="link" data-id="'
+ esc(link.key) + '" points="' + pts + '" fill="rgba(122, 31, 31, 0.06)"
stroke="' + stroke + '" stroke-width="' + sw + '" opacity="' + (isSel ? 1
: 0.75) + '" />';
        svg += '</g>';
    }
    });

    // ---- Live multi-selection lines ----
    if (state.multiSelection.length >= 2) {
        const sorted = state.multiSelection.slice().sort(function(a, b) {
return NODE_BY_ID[a].angle - NODE_BY_ID[b].angle; });
        if (sorted.length === 2) {
            const a = pos[sorted[0]], b = pos[sorted[1]];
            svg += '<line x1="' + a.x + '" y1="' + a.y + '" x2="' + b.x + '"
y2="' + b.y + '" stroke="var(--c-rhet)" stroke-width="2.5"
stroke-dasharray="3 3" pointer-events="none" opacity="0.95"/>';
        } else {
            const pts = sorted.map(function(id) { const p2 = pos[id]; return
p2.x + ',' + p2.y; }).join(' ');
            svg += '<polygon points="' + pts + '" fill="rgba(42, 58, 107, 0.08)"
stroke="var(--c-rhet)" stroke-width="2.5" stroke-dasharray="3 3"
pointer-events="none" opacity="0.95"/>';
        }
    }

    // ---- Branch lines for the selected concept's question bubbles ----
    svg += buildBranchLines();

    // ---- Leader line for links / multi-select (not single nodes) ----
    if ((state.selected && state.selected.type === 'link') ||
state.multiSelection.length >= 2) {
        let cx, cy, sourcePoints;
        if (state.multiSelection.length >= 2) {
            const nodes = state.multiSelection;

```

```

    let sx = 0, sy = 0;
    nodes.forEach(function(id) { sx += pos[id].x; sy += pos[id].y; });
    const ax = sx / nodes.length, ay = sy / nodes.length;
    const dx = ax - CENTER, dy = ay - CENTER;
    const dist = Math.sqrt(dx*dx + dy*dy) || 1;
    cx = ax + (dx/dist) * 160; cy = ay + (dy/dist) * 160;
    sourcePoints = nodes.map(function(id) { return pos[id]; });
  } else if (state.selected && state.selected.type === 'link') {
    const link = state.links.find(function(l) { return l.key ===
state.selected.id; });
    if (link) {
      let sx = 0, sy = 0;
      link.nodes.forEach(function(id) { sx += pos[id].x; sy +=
pos[id].y; });
      const ax = sx / link.nodes.length, ay = sy / link.nodes.length;
      const dx = ax - CENTER, dy = ay - CENTER;
      const dist = Math.sqrt(dx*dx + dy*dy) || 1;
      cx = ax + (dx/dist) * 160; cy = ay + (dy/dist) * 160;
      sourcePoints = link.nodes.map(function(id) { return pos[id]; });
    }
  }
  if (sourcePoints) {
    const ccx = Math.max(40, Math.min(VIEWBOX - 40, cx));
    const ccy = Math.max(60, Math.min(VIEWBOX - 40, cy));
    sourcePoints.forEach(function(p) {
      svg += '<line x1="' + p.x + '" y1="' + p.y + '" x2="' + ccx + '"
y2="' + ccy + '" stroke="var(--ink)" stroke-width="1.2"
stroke-dasharray="3 3" opacity="0.5" pointer-events="none"/>';
    });
  }
}

// (Center title and node labels now rendered in HTML overlay)

svg += '</svg>';
return svg;
}

```

```

function buildSphereOverlay() {
  let html = '<div class="sphere-overlay">';
  NODES_BASE.forEach(function(node) {
    const p = state.nodePositions[node.id];
    const lx = (p.x / VIEWBOX) * 100;
    const ly = (p.y / VIEWBOX) * 100;
    const isSel = state.selected && state.selected.type === 'node' &&

```

```

state.selected.id === node.id;
    const isMulti = state.multiSelection.indexOf(node.id) >= 0;
    const isBuilding = state.linkBuilding.indexOf(node.id) >= 0;
    const color = nodeColor(node.id);

    const classes = ['sphere-group', 'map-node-group'];
    if (isSel) classes.push('is-selected');
    if (isMulti || isBuilding) classes.push('is-multi');

    html += '<div class="' + classes.join(' ') + '" data-action="node"
data-id="' + esc(node.id) + '" style="left: ' + lx + '%; top: ' + ly + '%;
--sphere-color: ' + color + ';">';
    html += '<div class="sphere"><div class="sphere-label">' +
esc(node.label) + '</div></div>';
    html += '</div>';
    });
    // Center "Analysis Concepts" white node
    html += '<div class="center-node"><div
class="center-node-text">Analysis<br>Concepts</div></div>';
    html += '</div>';
    return html;
}

```

```

function buildLinkBuilder() {
    if (!state.linkMode) return '';
    const count = state.linkBuilding.length;
    let html = '<div class="link-builder">';
    html += '<span class="link-builder-label">Building link · ' + count + '
node' + (count === 1 ? '' : 's') + ' selected</span>';
    state.linkBuilding.forEach(function(id) {
        const n = NODE_BY_ID[id];
        html += '<span class="link-builder-tag" style="background: ' +
nodeColor(id) + '">' + esc(n.label) + '</span>';
    });
    html += '<span class="link-builder-buttons">';
    html += '<button class="panel-action" data-action="complete-link" ' +
(count < 2 ? 'disabled' : '') + '> Create link (' + count + ')</button>';
    html += '<button class="panel-action"
data-action="cancel-link">Cancel</button>';
    html += '</span>';
    html += '</div>';
    return html;
}

```

```

function buildConnectionHint() {

```

```

    let txt = '', cls = '';
    if (state.linkMode) {
        if (state.linkBuilding.length === 0) txt = '> Click nodes to add them
to your link';
        else if (state.linkBuilding.length === 1) txt = '> Click at least one
more node, then create the link';
        else txt = '> Ready to create – or keep adding more for a wider
hybrid';
    } else if (state.links.length > 0) {
        txt = state.links.length + ' connection' + (state.links.length > 1 ?
's' : '') + ' drawn';
    } else {
        txt = 'Click a node to see questions · drag to rearrange · shift+click
to multi-select';
        cls = 'idle';
    }
    return '<div class="connection-hint ' + cls + '">' + esc(txt) +
'</div>';
}

```

```

function getSelectedMeta() {
    // multi-selection takes priority when active
    if (state.multiSelection.length >= 2) {
        const sortedIds = state.multiSelection.slice().sort();
        const key = sortedIds.join('::');
        const sortedByAngle = state.multiSelection.slice().sort(function(a, b)
{ return NODE_BY_ID[a].angle - NODE_BY_ID[b].angle; });
        const nodes = sortedByAngle.map(function(id) { return NODE_BY_ID[id];
});
        const labels = nodes.map(function(n) { return n.label; }).join(' × ');
        const desc = 'A live hybrid of ' + nodes.length + ' concepts –
questions that require thinking across all of them at once.';
        return { isLink: true, isMulti: true, label: labels, color:
'var(--ink)', description: desc, nodes: nodes, key: key };
    }
    if (state.multiSelection.length === 1) {
        const n = NODE_BY_ID[state.multiSelection[0]];
        return { isLink: false, isMulti: true, label: n.label, color:
nodeColor(n.id), description: n.description, nodes: [n], key: n.id };
    }
    if (!state.selected) return null;
    if (state.selected.type === 'node') {
        const n = NODE_BY_ID[state.selected.id];
        return { isLink: false, label: n.label, color: nodeColor(n.id),
description: n.description, nodes: [n], key: n.id };
    }
}

```

```

    }
    const link = state.links.find(function(l) { return l.key ===
state.selected.id; });
    if (!link) return null;
    const sortedNodes = link.nodes.slice().sort(function(a, b) { return
NODE_BY_ID[a].angle - NODE_BY_ID[b].angle; });
    const nodes = sortedNodes.map(function(id) { return NODE_BY_ID[id]; });
    const labels = nodes.map(function(n) { return n.label; }).join(' × ');
    const desc = nodes.length === 2
        ? 'Where ' + nodes[0].label.toLowerCase() + ' meets ' +
nodes[1].label.toLowerCase() + ' - questions that exist only at their
intersection.'
        : 'A hybrid of ' + nodes.length + ' concepts - questions that require
thinking across all of them at once.';
    return { isLink: true, label: labels, color: 'var(--ink)', description:
desc, nodes: nodes, key: link.key };
}

function buildQuestionsSection() {
    const hasMulti = state.multiSelection.length > 0;
    if (!state.selected && !hasMulti) {
        return '<div class="questions-section"><div class="outline-empty"
style="padding: 20px 10px; font-size: 15px;">click a concept on the map to
begin · shift+click to multi-select</div></div>';
    }
    const m = getSelectedMeta();
    if (!m) return '<div class="questions-section"></div>';

    const key = m.key;
    const questions = state.questionCache[key];
    const isLoading = state.questionLoading[key];
    const error = state.questionError[key];

    let html = '<div class="questions-section">';
    html += '<div class="selected-tags">';
    if (m.isLink) {
        m.nodes.forEach(function(n, i) {
            if (i > 0) html += '<span class="tag-x">×</span>';
            html += '<span class="selected-tag" style="background: ' +
nodeColor(n.id) + '">' + esc(n.label) + '</span>';
        });
    } else {
        html += '<span class="selected-tag" style="background: ' + m.color +
'">' + esc(m.label) + '</span>';
    }
}

```

```

html += '<div style="flex:1"></div>';
if (m.isMulti && state.multiSelection.length >= 2) {
  html += '<button class="panel-action" data-action="save-multi">Ⓜ Save
as connection</button>';
  html += '<button class="panel-action" data-action="clear-multi"
style="margin-left: 8px;">× Clear multi-select</button>';
} else if (m.isMulti && state.multiSelection.length === 1) {
  html += '<button class="panel-action" data-action="clear-multi">×
Clear multi-select</button>';
} else if (state.selected && state.selected.type === 'link') {
  html += '<button class="panel-action" data-action="remove-link"
data-id="' + esc(state.selected.id) + '">× Remove link</button>';
}
if (questions && !isLoading) {
  html += '<button class="panel-action"
data-action="regenerate-questions" data-id="' + esc(key) + '"
style="margin-left: 8px;">↻ Regenerate</button>';
}
html += '</div>';
html += '<div class="selected-description">' + esc(m.description) +
'</div>';

if (isLoading) {
  html += '<div class="loading-msg">composing questions for your
prompt</div>';
} else if (error && !questions) {
  html += '<div class="error-msg">' + esc(error) + '</div>';
} else if (questions) {
  if (error) html += '<div style="font-family: var(--font-mono);
font-size: 9px; text-transform: uppercase; letter-spacing: 0.15em; color:
var(--ink-faded); margin-bottom: 12px;">' + esc(error) + '</div>';
  html += '<div class="questions-list">';
  questions.forEach(function(q, i) {
    const qKey = key + '::' + i;
    const isAdded = !!state.addedKeys[qKey];
    const border = m.isLink ? 'var(--ink)' : m.color;
    html += '<div class="question-item" style="border-left-color: ' +
border + '">';
    html += '<span class="question-number">' + ROMAN[i] + '</span>' +
esc(q);
    html += '<div><button class="question-add ' + (isAdded ? 'added' :
'') + '" ' + (isAdded ? 'disabled' : '') + ' data-action="add-outline"
data-qkey="' + esc(qKey) + '" data-qidx="' + i + '">' + (isAdded ? '✓
added to outline' : '→ add to outline') + '</button></div>';
    html += '</div>';
  });
}

```

```

    });
    html += '</div>';
  } else {
    html += '<div class="loading-msg" style="font-size: 14px;">preparing
questions</div>';
  }
  html += '</div>';
  return html;
}

// Compute the position of a question bubble for a given concept & index.
function questionBubblePosition(conceptId, idx, total) {
  const conceptPos = state.nodePositions[conceptId];
  const dx = conceptPos.x - CENTER;
  const dy = conceptPos.y - CENTER;
  const baseAngle = Math.atan2(dy, dx);
  const distance = 95;
  const arcSpread = Math.PI * 0.75; // ~135 degrees
  const t = total <= 1 ? 0.5 : idx / (total - 1);
  const angleOffset = (t - 0.5) * arcSpread;
  const angle = baseAngle + angleOffset;
  return {
    x: conceptPos.x + Math.cos(angle) * distance,
    y: conceptPos.y + Math.sin(angle) * distance,
    angle: angle
  };
}

// Compute the position of a deeper bubble branching off a question
bubble.
function deeperBubblePosition(parentQbPos, idx, total) {
  const distance = 70;
  const arcSpread = Math.PI * 0.55;
  const t = total <= 1 ? 0.5 : idx / (total - 1);
  const angleOffset = (t - 0.5) * arcSpread;
  const angle = parentQbPos.angle + angleOffset;
  return {
    x: parentQbPos.x + Math.cos(angle) * distance,
    y: parentQbPos.y + Math.sin(angle) * distance,
    angle: angle
  };
}

// Build the SVG connection lines between concept → questions →
deeper-questions

```

```

function buildBranchLines() {
  if (!state.selected || state.selected.type !== 'node') return '';
  const conceptId = state.selected.id;
  const questions = state.questionCache[conceptId];
  if (!questions) return '';
  const conceptPos = state.nodePositions[conceptId];
  const color = nodeColor(conceptId);
  let svg = '';
  questions.forEach(function(q, i) {
    const qbPos = questionBubblePosition(conceptId, i, questions.length);
    svg += '<line x1="' + conceptPos.x + '" y1="' + conceptPos.y + '"
x2="' + qbPos.x + '" y2="' + qbPos.y + '" stroke="' + color + '"
stroke-width="1.6" opacity="0.7" pointer-events="none"/>';
    const qKey = conceptId + '::' + i;
    const deepers = state.deeperQuestions[qKey];
    if (deepers) {
      deepers.forEach(function(dq, di) {
        const dPos = deeperBubblePosition(qbPos, di, deepers.length);
        svg += '<line x1="' + qbPos.x + '" y1="' + qbPos.y + '" x2="' +
dPos.x + '" y2="' + dPos.y + '" stroke="' + color + '" stroke-width="1"
opacity="0.5" stroke-dasharray="3 3" pointer-events="none"/>';
      });
    }
  });
  return svg;
}

// Build the HTML question bubbles + deeper bubbles + active panel
function buildQuestionBubbles() {
  if (!state.selected || state.selected.type !== 'node') return '';
  const conceptId = state.selected.id;
  const questions = state.questionCache[conceptId];
  const isLoading = state.questionLoading[conceptId];
  const error = state.questionError[conceptId];
  if (!questions && !isLoading && !error) return '';
  const color = nodeColor(conceptId);
  let html = '';

  // Loading / error state - show a small inline panel next to the concept
  if (isLoading || error) {
    const cPos = state.nodePositions[conceptId];
    const dx = cPos.x - CENTER, dy = cPos.y - CENTER;
    const dist = Math.sqrt(dx*dx + dy*dy) || 1;
    const px = cPos.x + (dx/dist) * 100, py = cPos.y + (dy/dist) * 100;
    const lx = Math.max(15, Math.min(85, (px / VIEWBOX) * 100));

```

```

    const ly = Math.max(15, Math.min(85, (py / VIEWBOX) * 100));
    html += '<div class="qpanel" style="left: ' + lx + '%; top: ' + ly +
'%; width: 200px;">';
    if (isLoading) html += '<div class="qpanel-loading">composing
questions...</div>';
    else html += '<div class="qpanel-error">' + esc(error) + '</div>';
    html += '</div>';
    return html;
}

if (!questions) return '';

// Render the 5 question bubbles
questions.forEach(function(q, i) {
    const qbPos = questionBubblePosition(conceptId, i, questions.length);
    const lx = (qbPos.x / VIEWBOX) * 100;
    const ly = (qbPos.y / VIEWBOX) * 100;
    const qKey = conceptId + '::' + i;
    const isActive = state.activeQuestionKey === qKey;
    const isAdded = !!state.addedKeys[qKey];
    let cls = 'question-bubble';
    if (isActive) cls += ' is-active';
    if (isAdded) cls += ' is-added';
    html += '<div class="' + cls + '" data-action="open-question"
data-qkey="' + esc(qKey) + '" data-qidx="' + i + '" style="left: ' + lx +
'%; top: ' + ly + '%; --bubble-color: ' + color + ';">';
    html += '<div class="question-bubble-inner">' + ROMAN[i] + '</div>';
    html += '</div>';

    // Deeper bubbles for this question if any exist
    const deepers = state.deeperQuestions[qKey];
    const dLoading = state.deeperLoading[qKey];
    const dError = state.deeperError[qKey];
    if (deepers) {
        deepers.forEach(function(dq, di) {
            const dPos = deeperBubblePosition(qbPos, di, deepers.length);
            const dlx = (dPos.x / VIEWBOX) * 100;
            const dly = (dPos.y / VIEWBOX) * 100;
            const dKey = qKey + '::' + di;
            const isDActive = state.activeDeeperKey === dKey;
            const isDAdded = !!state.addedKeys[dKey];
            let dcls = 'deeper-bubble';
            if (isDActive) dcls += ' is-active';
            if (isDAdded) dcls += ' is-added';
            html += '<div class="' + dcls + '" data-action="open-deeper"

```

```

data-dkey="'" + esc(dKey) + "'" style="left: ' + dlx + '%; top: ' + dly +
'%; --bubble-color: ' + color + ';">';
    html += '<div class="deeper-bubble-inner">' + (di + 1) + '</div>';
    html += '</div>';
    });
}
// Loading indicator for deeper questions
if (dLoading) {
    const dPos = { x: qbPos.x + Math.cos(qbPos.angle) * 70, y: qbPos.y +
Math.sin(qbPos.angle) * 70 };
    const dlx = (dPos.x / VIEWBOX) * 100;
    const dly = (dPos.y / VIEWBOX) * 100;
    html += '<div class="qpanel" style="left: ' + dlx + '%; top: ' + dly
+ '%; width: 170px;"><div class="qpanel-loading">going
deeper...</div></div>';
    }
});

// Active question panel (full text + actions)
if (state.activeQuestionKey) {
    const parts = state.activeQuestionKey.split('::');
    const cId = parts[0];
    const i = parseInt(parts[1], 10);
    if (cId === conceptId && questions[i]) {
        const qbPos = questionBubblePosition(cId, i, questions.length);
        const dx = qbPos.x - CENTER, dy = qbPos.y - CENTER;
        const dist = Math.sqrt(dx*dx + dy*dy) || 1;
        const px = qbPos.x + (dx/dist) * 110, py = qbPos.y + (dy/dist) *
110;
        const plx = Math.max(20, Math.min(80, (px / VIEWBOX) * 100));
        const ply = Math.max(20, Math.min(80, (py / VIEWBOX) * 100));
        const qKey = state.activeQuestionKey;
        const isAdded = !!state.addedKeys[qKey];
        const dLoading = state.deeperLoading[qKey];
        html += '<div class="qpanel" style="left: ' + plx + '%; top: ' + ply
+ '%;">';
        html += '<button class="qpanel-close"
data-action="close-question">x</button>';
        html += '<div class="qpanel-meta">' +
esc(NODE_BY_ID[conceptId].label) + ' · question ' + ROMAN[i] + '</div>';
        html += '<div class="qpanel-text">' + esc(questions[i]) + '</div>';
        html += '<div class="qpanel-actions">';
        html += '<button class="panel-action" ' + (isAdded ? 'disabled' :
'') + ' data-action="add-outline" data-qkey="' + esc(qKey) + '"
data-qidx="' + i + '">' + (isAdded ? '✓ added' : '→ add to outline') +

```

```

'</button>';
    html += '<button class="panel-action" ' + (dLoading ? 'disabled' :
'' ) + ' data-action="ask-deeper" data-qkey="' + esc(qKey) + '">' +
(dLoading ? 'generating...' : '→ Ask deeper') + '</button>';
    html += '</div>';
    html += '</div>';
}
}

// Active deeper-question panel
if (state.activeDeeperKey) {
    const parts = state.activeDeeperKey.split('::');
    const cId = parts[0], qi = parseInt(parts[1], 10), di =
parseInt(parts[2], 10);
    if (cId === conceptId) {
        const parentQKey = cId + '::' + qi;
        const deepers = state.deeperQuestions[parentQKey];
        if (deepers && deepers[di]) {
            const qbPos = questionBubblePosition(cId, qi, questions.length);
            const dPos = deeperBubblePosition(qbPos, di, deepers.length);
            const dx = dPos.x - CENTER, dy = dPos.y - CENTER;
            const dist = Math.sqrt(dx*dx + dy*dy) || 1;
            const px = dPos.x + (dx/dist) * 90, py = dPos.y + (dy/dist) * 90;
            const plx = Math.max(20, Math.min(80, (px / VIEWBOX) * 100));
            const ply = Math.max(20, Math.min(80, (py / VIEWBOX) * 100));
            const dKey = state.activeDeeperKey;
            const isAdded = !!state.addedKeys[dKey];
            html += '<div class="qpanel" style="left: ' + plx + '%; top: ' +
ply + '%;">';
            html += '<button class="qpanel-close"
data-action="close-deeper">x</button>';
            html += '<div class="qpanel-meta">' +
esc(NODE_BY_ID[conceptId].label) + ' · deeper · ' + (di + 1) + '</div>';
            html += '<div class="qpanel-text">' + esc(deepers[di]) + '</div>';
            html += '<div class="qpanel-actions">';
            html += '<button class="panel-action" ' + (isAdded ? 'disabled' :
'' ) + ' data-action="add-deeper-outline" data-dkey="' + esc(dKey) + '">' +
(isAdded ? '✓ added' : '→ add to outline') + '</button>';
            html += '</div>';
            html += '</div>';
        }
    }
}
}

return html;

```

```

}

// Stub: keep existing callout pathway for links / multi-select since
those still want a list view
function buildCallout() {
  // Show only for link selection or multi-selection (not single-node,
  which uses bubbles)
  if (state.selected && state.selected.type === 'node' &&
state.multiSelection.length === 0) return '';
  if (!state.selected && state.multiSelection.length === 0) return '';
  const m = getSelectedMeta();
  if (!m) return '';
  const key = m.key;
  const questions = state.questionCache[key];
  const isLoading = state.questionLoading[key];
  const error = state.questionError[key];

  let cx, cy;
  let sx = 0, sy = 0;
  m.nodes.forEach(function(n) { const p = state.nodePositions[n.id]; sx +=
p.x; sy += p.y; });
  const ax = sx / m.nodes.length, ay = sy / m.nodes.length;
  const dx = ax - CENTER, dy = ay - CENTER;
  const dist = Math.sqrt(dx*dx + dy*dy) || 1;
  cx = ax + (dx/dist) * 160;
  cy = ay + (dy/dist) * 160;
  cx = Math.max(40, Math.min(VIEWBOX - 40, cx));
  cy = Math.max(60, Math.min(VIEWBOX - 40, cy));
  const cxPct = (cx / VIEWBOX) * 100;
  const cyPct = (cy / VIEWBOX) * 100;

  let html = '<div class="qpanel" style="left: ' + cxPct + '%; top: ' +
cyPct + '%; width: 320px;">';
  html += '<button class="qpanel-close"
data-action="close-callout">x</button>';
  html += '<div class="qpanel-meta">';
  html += m.nodes.map(function(n) { return esc(n.label); }).join(' × ');
  html += '</div>';
  html += '<div style="font-family: var(--font-display); font-style:
italic; font-size: 13px; color: var(--ink-soft); line-height: 1.45;
margin-bottom: 12px;">' + esc(m.description) + '</div>';

  if (isLoading) {
    html += '<div class="qpanel-loading">composing questions...</div>';
  } else if (error && !questions) {

```

```

    html += '<div class="qpanel-error">' + esc(error) + '</div>';
  } else if (questions) {
    questions.forEach(function(q, i) {
      const qKey = key + '::' + i;
      const isAdded = !!state.addedKeys[qKey];
      html += '<div style="padding: 9px 11px; background: var(--bg);
border-left: 3px solid var(--ink); font-size: 14px; line-height: 1.4;
margin-bottom: 7px; display: flex; gap: 8px;">';
      html += '<span style="font-family: var(--font-display); font-style:
italic; color: var(--ink-faded); font-size: 12px; flex-shrink: 0;">' +
ROMAN[i] + '.</span>';
      html += '<span style="flex: 1;">' + esc(q) + '</span>';
      html += '<button style="background:none; border:none; padding:0;
font-family: var(--font-mono); font-size: 10px; text-transform: uppercase;
letter-spacing: 0.12em; color: ' + (isAdded ? 'var(--c-time)' :
'var(--ink-faded)') + '; cursor: ' + (isAdded ? 'default' : 'pointer') +
';" ' + (isAdded ? 'disabled' : '') + ' data-action="add-outline"
data-qkey="' + esc(qKey) + '" data-qidx="' + i + '">' + (isAdded ? '✓' :
'→ add') + '</button>';
      html += '</div>';
    });
  }
  html += '<div class="qpanel-actions" style="border-top: 1px solid
var(--rule-soft); padding-top: 10px; margin-top: 6px;">';
  if (questions && !isLoading) html += '<button class="panel-action"
data-action="regenerate-questions" data-id="' + esc(key) + '">↻
Regenerate</button>';
  if (m.isMulti && state.multiSelection.length >= 2) {
    html += '<button class="panel-action" data-action="save-multi">💾
Save</button>';
    html += '<button class="panel-action" data-action="clear-multi">✕
Clear</button>';
  } else if (state.selected && state.selected.type === 'link') {
    html += '<button class="panel-action" data-action="remove-link"
data-id="' + esc(state.selected.id) + '">✕ Remove link</button>';
  }
  html += '</div></div>';
  return html;
}

function buildMapPanel() {
  let html = '<div class="panel">';
  html += '<div class="panel-label"><span>Container I</span>';
  html += '<div style="display: flex; gap: 8px;">';
  if (state.linkMode) {

```

```

    html += '<button class="panel-action active"
data-action="toggle-link-mode">Cancel linking</button>';
  } else {
    html += '<button class="panel-action" data-action="toggle-link-mode">>
Link concepts</button>';
  }
  html += '<button class="panel-action" data-action="reset-positions"
title="Reset nodes to hexagon">△ Reset positions</button>';
  html += '</div>';
  html += '</div>';
  html += '<div class="panel-title">The Map</div>';
  if (state.prompt) {
    html += '<div class="anchored-prompt">' + esc(state.prompt) +
'</div>';
  }
  html += buildLinkBuilder();
  html += '<div class="map-perspective"><div class="map-wrap"
id="map-wrap">' + buildSVG() + buildSphereOverlay() +
buildQuestionBubbles() + buildCallout() + '</div></div>';
  html += buildConnectionHint();
  html += '</div>';
  return html;
}

```

```

function buildOutlinePanel() {
  let html = '<div class="panel">';
  html += '<div class="panel-label"><span>Container II</span>';
  if (state.outline.length > 0) html += '<button class="panel-action"
data-action="copy-outline">📋 Copy as Markdown</button>';
  html += '</div>';
  html += '<div class="panel-title">The Outline</div>';
  if (state.outline.length === 0) {
    html += '<div class="outline-empty">The questions you select <br>will
land here as your <br>analytical spine.</div>';
  } else {
    state.outline.forEach(function(item, idx) {
      html += '<div class="outline-item" style="border-left-color: ' +
item.color + '>';
      html += '<div class="outline-source" style="color: ' + item.color +
'><span class="roman">' + ROMAN[idx] + '</span><span>· ' +
esc(item.sourceLabel) + '</span></div>';
      html += '<div class="outline-question-text">' + esc(item.question) +
'</div>';
      html += '<textarea id="notes-' + item.id + '" class="outline-notes"
placeholder="Your thinking – what you notice, the evidence forming in your

```

```

mind, the counterpoint..."></textarea>';
    html += '<div class="outline-controls">';
    html += '<button class="outline-mini-btn" data-action="move-up"
data-id="' + item.id + '" ' + (idx === 0 ? 'disabled' : '') + '>↑ move
up</button>';
    html += '<button class="outline-mini-btn" data-action="move-down"
data-id="' + item.id + '" ' + (idx === state.outline.length - 1 ?
'disabled' : '') + '>↓ move down</button>';
    html += '<div style="flex:1"></div>';
    html += '<button class="outline-mini-btn remove"
data-action="remove-outline" data-id="' + item.id + '">× remove</button>';
    html += '</div></div>';
    });
  }
  html += '</div>';
  return html;
}

```

```

// =====
// RENDER
// =====

```

```

function render() {
  // Preserve outline notes from DOM
  state.outline.forEach(function(item) {
    const ta = $('notes-' + item.id);
    if (ta && ta.value !== undefined) item.notes = ta.value;
  });

  const html = ''
    + '<div class="app">'
    +   buildHeader()
    +   buildPromptSection()
    +   '<div class="main-grid">' + buildMapPanel() + buildOutlinePanel()
+ '</div>'
    +   '<div class="colophon">- questions, not answers -</div>'
    + '</div>';

```

```

$('app').innerHTML = html;

```

```

// Restore textarea values
const promptTa = $('prompt-textarea');
if (promptTa) promptTa.value = state.promptDraft;
state.outline.forEach(function(item) {
  const ta = $('notes-' + item.id);

```

```

        if (ta) ta.value = item.notes || '';
    });

    attachEvents();
}

// =====
// ACTIONS
// =====

function toggleTheme() {
    state.theme = state.theme === 'light' ? 'dark' : 'light';
    document.documentElement.setAttribute('data-theme', state.theme);
    render();
}

function begin() {
    const draft = state.promptDraft.trim();
    if (draft.length < 1) return;
    state.prompt = draft;
    // Clear cached questions since prompt changed – they need regeneration
    state.questionCache = {};
    state.questionError = {};
    render();
    // If something is selected, fetch new questions
    if (state.selected) {
        const m = getSelectedMeta();
        if (m) fetchQuestionsFor(m.key);
    }
}

function reset() {
    state.promptDraft = '';
    state.prompt = '';
    state.outline = [];
    state.links = [];
    state.selected = null;
    state.linkMode = false;
    state.linkBuilding = [];
    state.multiSelection = [];
    state.nodePositions = defaultNodePositions();
    state.addedKeys = {};
    state.questionCache = {};
    state.questionLoading = {};
    state.questionError = {};

```

```

    render();
}

function toggleLinkMode() {
    state.linkMode = !state.linkMode;
    state.linkBuilding = [];
    render();
}

function completeLinkBuilding() {
    if (state.linkBuilding.length < 2) return;
    const sortedIds = state.linkBuilding.slice().sort();
    const key = sortedIds.join('::');
    const exists = state.links.some(function(l) { return l.key === key; });
    if (!exists) state.links.push({ key: key, nodes: sortedIds });
    state.selected = { type: 'link', id: key };
    state.linkBuilding = [];
    state.linkMode = false;
    render();
    fetchQuestionsFor(key);
}

function cancelLinkBuilding() {
    state.linkBuilding = [];
    state.linkMode = false;
    render();
}

function handleNodeClick(nodeId, shiftKey) {
    if (state.linkMode) {
        const idx = state.linkBuilding.indexOf(nodeId);
        if (idx >= 0) state.linkBuilding.splice(idx, 1);
        else state.linkBuilding.push(nodeId);
        render();
        return;
    }
    if (shiftKey) {
        // Shift+click → toggle node in/out of multi-selection
        const idx = state.multiSelection.indexOf(nodeId);
        if (idx >= 0) state.multiSelection.splice(idx, 1);
        else state.multiSelection.push(nodeId);
        state.selected = null; // multi takes priority
        render();
        if (state.multiSelection.length >= 2) {
            const key = state.multiSelection.slice().sort().join('::');

```

```

        fetchQuestionsFor(key);
    } else if (state.multiSelection.length === 1) {
        fetchQuestionsFor(state.multiSelection[0]);
    }
    return;
}
// Plain click → clear multi, single-select
state.multiSelection = [];
state.selected = { type: 'node', id: nodeId };
state.activeQuestionKey = null;
state.activeDeeperKey = null;
render();
fetchQuestionsFor(nodeId);
}

function clearMultiSelection() {
    state.multiSelection = [];
    state.selected = null;
    render();
}

function saveMultiAsLink() {
    if (state.multiSelection.length < 2) return;
    const sortedIds = state.multiSelection.slice().sort();
    const key = sortedIds.join('::');
    const exists = state.links.some(function(l) { return l.key === key; });
    if (!exists) state.links.push({ key: key, nodes: sortedIds });
    state.multiSelection = [];
    state.selected = { type: 'link', id: key };
    render();
}

function resetNodePositions() {
    state.nodePositions = defaultNodePositions();
    render();
}

function closeCallout() {
    state.selected = null;
    state.multiSelection = [];
    state.activeQuestionKey = null;
    state.activeDeeperKey = null;
    render();
}

```

```

function openQuestionBubble(qKey) {
  state.activeQuestionKey = qKey;
  state.activeDeeperKey = null;
  render();
}

function closeQuestionBubble() {
  state.activeQuestionKey = null;
  state.activeDeeperKey = null;
  render();
}

function openDeeperBubble(dKey) {
  state.activeDeeperKey = dKey;
  // Keep the parent question panel open too (don't clear
  activeQuestionKey)
  render();
}

function closeDeeperBubble() {
  state.activeDeeperKey = null;
  render();
}

async function askDeeperQuestions(qKey) {
  if (state.deeperLoading[qKey]) return;
  const parts = qKey.split('::');
  const conceptId = parts[0];
  const qIdx = parseInt(parts[1], 10);
  const baseQuestions = state.questionCache[conceptId];
  if (!baseQuestions || !baseQuestions[qIdx]) return;
  const originalQuestion = baseQuestions[qIdx];
  const concept = NODE_BY_ID[conceptId];

  state.deeperLoading[qKey] = true;
  state.deeperError[qKey] = '';
  render();

  // Build a prompt for deeper, more specific Socratic questions
  const apiPrompt =
    'You are a Socratic tutor for an IB Literature student. They are
  exploring the lens of "' + concept.label + '" ' +
    'and currently working through this question:\n\n"' + originalQuestion
  + '"\n\n' +
    (state.prompt ? ('Their broader essay focus: "' + state.prompt +

```

```

'\n\n') : '' ) +
  'Generate exactly 3 follow-up Socratic questions that go DEEPER than
the question above – more specific, more probing, ' +
  'each one extending or sharpening that exact question. They should not
just rephrase it. They should push the student ' +
  'toward a more particular observation or claim.\n\n' +
  'Return ONLY a JSON array of 3 short question strings (no preamble, no
markdown, no keys).';

let result = null;
let errMsg = '';
try {
  if (typeof fetch === 'function' && typeof window !== 'undefined' &&
window.location && window.location.hostname.indexOf('claude.ai') >= 0) {
    const resp = await fetch('https://api.anthropic.com/v1/messages', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        model: 'claude-sonnet-4-5',
        max_tokens: 600,
        messages: [{ role: 'user', content: apiPrompt }]
      })
    });
    const data = await resp.json();
    const text = (data.content || []).map(function(c) { return c.text ||
''; }).join('').trim();
    const cleaned = text.replace(/```json|```/g, '').trim();
    const parsed = JSON.parse(cleaned);
    if (Array.isArray(parsed) && parsed.length) result = parsed.slice(0,
3);
  }
} catch (e) {
  errMsg = 'Could not generate deeper questions';
}

// Fallback: synthesize deeper questions from a small bank of stems if
API unavailable
if (!result) {
  const stems = [
    'And what would change about this if it were taken to its most
concrete instance?',
    'What would have to be true of the text for the answer to be
otherwise?',
    'What is the smallest, most specific piece of the text that proves
your case?'
  ]
}

```

```

    ];
    result = stems;
    if (!errMsg) errMsg = 'Offline · using fallback prompts';
}

state.deeperQuestions[qKey] = result;
state.deeperLoading[qKey] = false;
if (errMsg) state.deeperError[qKey] = errMsg;
render();
}

function addDeeperToOutline(dKey) {
    // dKey format: "conceptId::qi::di"
    const parts = dKey.split('::');
    const conceptId = parts[0];
    const qi = parseInt(parts[1], 10);
    const di = parseInt(parts[2], 10);
    const parentQKey = conceptId + '::' + qi;
    const deepers = state.deeperQuestions[parentQKey];
    if (!deepers || !deepers[di]) return;
    const question = deepers[di];
    const concept = NODE_BY_ID[conceptId];
    const sourceLabel = concept.label + ' · deeper';
    const color = nodeColor(conceptId);
    state.outline.push({
        id: 'o' + Date.now() + '-' + Math.floor(Math.random() * 10000),
        question: question, sourceLabel: sourceLabel, color: color, notes: '',
qKey: dKey
    });
    state.addedKeys[dKey] = true;
    render();
}

function handleLinkClick(key) {
    if (state.linkMode) return;
    state.selected = { type: 'link', id: key };
    render();
    fetchQuestionsFor(key);
}

function removeLink(key) {
    state.links = state.links.filter(function(l) { return l.key !== key; });
    if (state.selected && state.selected.type === 'link' &&
state.selected.id === key) {
        state.selected = null;
    }
}

```

```

    }
    render();
}

function regenerateQuestions(key) {
    delete state.questionCache[key];
    delete state.questionError[key];
    render();
    fetchQuestionsFor(key);
}

function addToOutline(qKey, qIdx) {
    const m = getSelectedMeta();
    if (!m) return;
    const questions = state.questionCache[m.key];
    if (!questions || !questions[qIdx]) return;
    const question = questions[qIdx];
    const sourceLabel = m.isLink ? m.nodes.map(function(n) { return n.label;
}).join(' × ') : m.label;
    const color = m.isLink ? 'var(--ink)' : m.color;
    state.outline.push({
        id: 'o' + Date.now() + '-' + Math.floor(Math.random() * 10000),
        question: question, sourceLabel: sourceLabel, color: color, notes: '',
qKey: qKey
    });
    state.addedKeys[qKey] = true;
    render();
}

function removeFromOutline(id) {
    const item = state.outline.find(function(i) { return i.id === id; });
    state.outline = state.outline.filter(function(i) { return i.id !== id;
});
    if (item) delete state.addedKeys[item.qKey];
    render();
}

function moveOutline(id, dir) {
    const idx = state.outline.findIndex(function(i) { return i.id === id;
});
    if (idx < 0) return;
    const ni = idx + dir;
    if (ni < 0 || ni >= state.outline.length) return;
    const tmp = state.outline[idx];
    state.outline[idx] = state.outline[ni];

```

```

    state.outline[ni] = tmp;
    render();
}

function exportOutline() {
    state.outline.forEach(function(item) {
        const ta = $('notes-' + item.id);
        if (ta) item.notes = ta.value;
    });
    const lines = ['# Analytical Outline', ''];
    if (state.prompt) lines.push('**Prompt:** ' + state.prompt, '');
    state.outline.forEach(function(it, i) {
        lines.push('## ' + (i + 1) + '. ' + it.sourceLabel);
        lines.push('');
        lines.push('> ' + it.question);
        lines.push('');
        if (it.notes && it.notes.trim()) {
            lines.push(it.notes.trim());
            lines.push('');
        }
    });
    const text = lines.join('\n');
    if (navigator.clipboard && navigator.clipboard.writeText) {
        navigator.clipboard.writeText(text).then(
            function() { alert('Outline copied to clipboard.');
            function() { window.prompt('Copy this:', text); }
        );
    } else {
        window.prompt('Copy this:', text);
    }
}

// =====
// EVENT BINDING
// =====

function attachEvents() {
    $('app').onclick = function(e) {
        const target = e.target.closest('[data-action]');
        if (!target) return;
        const action = target.dataset.action;
        const id = target.dataset.id;
        switch (action) {
            case 'toggle-theme': toggleTheme(); break;
            case 'begin': begin(); break;
        }
    }
}

```

```

    case 'reset': reset(); break;
    case 'toggle-link-mode': toggleLinkMode(); break;
    case 'complete-link': completeLinkBuilding(); break;
    case 'cancel-link': cancelLinkBuilding(); break;
    case 'node': handleNodeClick(id, e.shiftKey); break;
    case 'link': handleLinkClick(id); break;
    case 'remove-link': removeLink(id); break;
    case 'close-callout': closeCallout(); break;
    case 'open-question': openQuestionBubble(target.dataset.qkey);
break;
    case 'close-question': closeQuestionBubble(); break;
    case 'open-deeper': openDeeperBubble(target.dataset.dkey); break;
    case 'close-deeper': closeDeeperBubble(); break;
    case 'ask-deeper': askDeeperQuestions(target.dataset.qkey); break;
    case 'add-deeper-outline': addDeeperToOutline(target.dataset.dkey);
break;
    case 'clear-multi': clearMultiSelection(); break;
    case 'save-multi': saveMultiAsLink(); break;
    case 'reset-positions': resetNodePositions(); break;
    case 'regenerate-questions': regenerateQuestions(id); break;
    case 'add-outline': addToOutline(target.dataset.qkey,
parseInt(target.dataset.qidx, 10)); break;
    case 'move-up': moveOutline(id, -1); break;
    case 'move-down': moveOutline(id, 1); break;
    case 'remove-outline': removeFromOutline(id); break;
    case 'copy-outline': exportOutline(); break;
  }
};

const promptTa = $('prompt-textarea');
if (promptTa) {
  promptTa.oninput = function(e) { state.promptDraft = e.target.value;
};
  promptTa.onkeydown = function(e) {
    if ((e.metaKey || e.ctrlKey) && e.key === 'Enter') {
      e.preventDefault();
      begin();
    }
  };
}

state.outline.forEach(function(item) {
  const ta = $('notes-' + item.id);
  if (ta) {
    ta.oninput = function(e) {

```

```

        const it = state.outline.find(function(i) { return i.id ===
item.id; });
        if (it) it.notes = e.target.value;
    };
}
});

attachMapRotation();
}

// =====
// 3D MAP ROTATION
// =====

const rotation = { x: -15, y: 20 }; // initial subtle tilt so 3D is
visible
let dragState = null;
let didDrag = false;
let windowListenersAttached = false;

function applyRotation() {
    // Map is flat now – no rotation applied
}

function attachMapRotation() {
    // Map is flat now – rotation disabled. Still attach node dragging.
    attachNodeDragging();
}

// =====
// NODE DRAGGING – moves individual nodes on the SVG plane
// =====

let nodeDragState = null;
let nodeDidDrag = false;
let nodeWindowListenersAttached = false;

function attachNodeDragging() {
    const groups = document.querySelectorAll('.map-node-group');
    groups.forEach(function(g) {
        g.addEventListener('mousedown', startNodeDrag);
        g.addEventListener('touchstart', function(e) {
            if (e.touches.length !== 1) return;
            const t = e.touches[0];
            startNodeDrag({ clientX: t.clientX, clientY: t.clientY,

```

```

currentTarget: g, preventDefault: function() {}, stopPropagation:
function() {} });
    }, { passive: true });
    // Suppress click after drag
    g.addEventListener('click', function(e) {
        if (nodeDidDrag) { e.stopPropagation(); e.preventDefault(); }
    }, true);
});

if (!nodeWindowListenersAttached) {
    window.addEventListener('mousemove', function(e) {
moveNodeDrag(e.clientX, e.clientY); });
    window.addEventListener('mouseup', endNodeDrag);
    window.addEventListener('touchmove', function(e) {
        if (e.touches.length === 1 && nodeDragState) {
            moveNodeDrag(e.touches[0].clientX, e.touches[0].clientY);
            e.preventDefault();
        }
    }, { passive: false });
    window.addEventListener('touchend', endNodeDrag);
    nodeWindowListenersAttached = true;
}
}

function svgScale() {
    // Compute pixels-per-SVG-unit for the current map
    const svg = document.querySelector('.map-svg');
    if (!svg) return 1;
    const rect = svg.getBoundingClientRect();
    // SVG is square with viewBox of VIEWBOX (440)
    return rect.width / VIEWBOX;
}

function startNodeDrag(e) {
    const g = e.currentTarget;
    const nodeId = g.getAttribute('data-id');
    if (!nodeId || !state.nodePositions[nodeId]) return;
    nodeDragState = {
        nodeId: nodeId,
        startX: e.clientX,
        startY: e.clientY,
        initX: state.nodePositions[nodeId].x,
        initY: state.nodePositions[nodeId].y
    };
    nodeDidDrag = false;

```

```

    if (e.stopPropagation) e.stopPropagation();
    if (e.preventDefault) e.preventDefault();
}

function moveNodeDrag(clientX, clientY) {
    if (!nodeDragState) return;
    const scale = svgScale() || 1;
    const dx = (clientX - nodeDragState.startX) / scale;
    const dy = (clientY - nodeDragState.startY) / scale;
    if (Math.abs(dx) > 2 || Math.abs(dy) > 2) nodeDidDrag = true;
    if (!nodeDidDrag) return;
    // Update position; clamp inside the viewBox with a small margin
    const margin = 32;
    const nx = Math.max(margin, Math.min(VIEWBOX - margin,
nodeDragState.initX + dx));
    const ny = Math.max(margin, Math.min(VIEWBOX - margin,
nodeDragState.initY + dy));
    state.nodePositions[nodeDragState.nodeId] = { x: nx, y: ny };
    // Update SVG in place without a full re-render to keep the drag smooth
    updateMapSVG();
}

function endNodeDrag() {
    if (!nodeDragState) return;
    nodeDragState = null;
    setTimeout(function() { nodeDidDrag = false; }, 50);
}

function updateMapSVG() {
    // Replace SVG, sphere overlay, and callout inside #map-wrap
    const w = $('map-wrap');
    if (w) w.innerHTML = buildSVG() + buildSphereOverlay() +
buildQuestionBubbles() + buildCallout();
    // Re-attach node drag handlers (window listeners stay)
    const groups = document.querySelectorAll('.map-node-group');
    groups.forEach(function(g) {
        g.addEventListener('mousedown', startNodeDrag);
        g.addEventListener('touchstart', function(e) {
            if (e.touches.length !== 1) return;
            const t = e.touches[0];
            startNodeDrag({ clientX: t.clientX, clientY: t.clientY,
currentTarget: g, preventDefault: function() {}, stopPropagation:
function() {} });
        }, { passive: true });
        g.addEventListener('click', function(e) {

```

```
        if (nodeDidDrag) { e.stopPropagation(); e.preventDefault(); }
    }, true);
});
}

// =====
// INIT
// =====

render();
</script>
</body>
</html>
```